Object Mapping

Object Mapping

`gemstone-js` supports object mapping, but it keeps the GemStone object model explicit. The library maps JavaScript values and TypeScript types to GemStone calls and object handles; it does not automatically hydrate GemStone objects into mutable JavaScript class instances.

That distinction is deliberate. GemStone objects have identity, session affinity, transaction visibility, and remote selector-send semantics. Hiding those behind plain JavaScript property reads would make common workflows look simpler while making correctness harder to reason about.

Mapping Layers

Use these layers according to how much object identity you need to preserve:

- **Value marshalling**: `performWith()`, `performValueWith()`, `ManagedOop.send()`, arrays, and dictionaries convert common JavaScript values into GemStone objects and marshal simple results back.
- **Raw identity**: `Oop` is the branded `bigint` handle for a GemStone object. Use raw OOP APIs when you need exact identity or want to avoid result marshalling.
- **Retained typed handles**: `TypedOop<T>` keeps a GemStone object retained for the session and gives TypeScript a compile-time witness for the expected domain type.
- **Class references**: `Session.classRef<T>()` caches class lookup and exposes class-side sends, allocation, and object-returning sends.
- **Mapped object proxies**: `mappedObject()` wraps a retained handle with async property-style methods such as `booking.status()` and `booking.setStatus("confirmed")`, while keeping remote calls explicit.
- **Transparent object proxies**: `transparentObject()` wraps a retained handle with awaitable properties such as `await booking.status`, callable selector accessors, queued assignment writes, optional read caching, and identity-aware proxy reuse through `TransparentObjectMapper`.
- **Smalltalk-style bridge**: `smalltalkBridge()` resolves GemStone globals lazily from JavaScript properties and maps names such as `new_` or `at_put_` to Smalltalk selectors such as `new:` and `at:put:`.
- **Generated wrappers**: codegen manifests and decorated source emit typed functions that call GemStone selectors and return values, raw OOPs, or retained typed handles.
- **Dictionary payloads**: `objectToDictionaryArgument()` converts plain objects or class instances into explicit `StringKeyValueDictionary` payloads before persistence or selector sends.
- **Value converters**: `ValueConverterRegistry` lets sessions marshal richer scalar values such as `Date` without changing the default persistence contract.

Choosing a Mapping Style

Start with the smallest layer that preserves the semantics you need:

- Call one selector with simple values: Use `performWith()` or `classRef().send()`. The result is a marshalled JavaScript value.
- Keep GemStone identity: Use `TypedOop<T>`. The result is a retained object handle.
- Make selector sends read like domain methods: Use `mappedObject()`. The result is an async proxy around a retained handle.
- Make selector reads look like object properties: Use `transparentObject()`. The result is an awaitable proxy with queued write support.

- Explore globals and class-side selectors dynamically: Use `smalltalkBridge()`. The result is a Python-style bridge with lazy global proxies and underscore-to-colon selector names.
- Share selector contracts across a codebase: Use a codegen manifest or decorators. The result is reviewable typed wrapper source.
- Persist keyed payload data: Use `GsDict`, `PersistentRoot`, or dictionary helpers. The result is explicit dictionary values or handles.
- Return UI/API payloads: Use `$snapshot()`, generated `snapshot()` wrappers, or bounded readback. The result is plain JavaScript data.

The usual progression is:

1. Use `classRef().sendObject()` or generated wrapper functions to find the GemStone object.
2. Wrap the result in `mappedObject()`, `transparentObject()`, or a generated `*Ref` class when the selector set is reused.
3. Use snapshots or dictionaries only at process boundaries such as HTTP responses, job payloads, logs, and UI panes.
4. Commit generated wrappers and mapping manifests so selector behavior can be reviewed like normal source.

Typed Object Handles

Use `TypedOop<T>` when a GemStone selector returns a live object and you want TypeScript to remember the expected domain shape.

```
import { Session, type TypedOop } from "gemstone-js";

interface Booking {
  id: string;
  status: string;
}

const session = await Session.connect();
const booking: TypedOop<Booking> = await session
  .classRef<Booking>("Booking")
  .sendObject("find:", "B-1001");

const status = await booking.send<string>("status");
await booking.send("status:", "confirmed");
await session.commit();
await booking.release();
```

This is object mapping through a retained remote handle. The `Booking` type is a compile-time witness for the handle; it is not a hydrated local instance. Selectors still make remote calls, and writes still follow the active GemStone transaction.

Opt-In Transparent Mapping

`mappedObject()` is the safe transparent layer. It makes selector sends feel closer to a local object API, but every remote operation is still an async method call.

```
import { Session, mappedObject, type TypedOop } from "gemstone-js";

interface Booking {
  id: string;
  status: string;
}

type BookingMapping = {
  setStatus(status: string): Promise<unknown>;
  customer(): Promise<TypedOop<{ name: string }>>;
```

```
};

const session = await Session.connect();
const object = await session
  .classRef<Booking>("BookingRepository")
  .sendObject("find:", "B-1001");

const booking = mappedObject<Booking, BookingMapping>(object, {
  selectors: {
    id: "id",
    status: "status",
  },
  setters: {
    setStatus: "status:",
  },
  objectSelectors: {
    customer: "customer",
  },
  snapshot: ["id", "status"],
});

const before = await booking.status();
await booking.setStatus("confirmed");
const customer = await booking.customer();
const payload = await booking.$snapshot();
```

The transparent part is the method name, not the execution model. The call still crosses a GemStone session boundary:

```
await booking.status();          // sends status
await booking.priority(3);       // sends priority:
await booking.setStatus("held"); // sends status:
```

The helper reserves `$`-prefixed operations for explicit remote controls:

- `$object`, `$session`, and `$oop` expose the retained handle boundary.
- `$send()`, `$sendOop()`, and `$sendObject()` send explicit selectors.
- `$set("status", "confirmed")` sends `status:` when you do not want a typed setter method.
- `$snapshot()` reads a configured or supplied field list into plain payload data.
- `$inspect()`, `$dump()`, `$printString()`, and `$release()` delegate to the retained handle.

Unknown non-`$` properties become async selector functions. A zero-argument method such as `booking.status()` sends `status`. A one-argument method such as `booking.priority(3)` sends `priority:`. Multi-argument selectors must be configured explicitly because JavaScript method names cannot infer Smalltalk keyword shape safely.

Awaitable Transparent Mapping

`transparentObject()` is the closest JavaScript equivalent to gemstone-py's `booking.proxy().status` model. JavaScript cannot synchronously block a property read on a remote GemStone selector, so the transparent read shape is `await booking.status`.

```
import {
  transparentObject,
  type Oop,
  type TypedOop,
} from "gemstone-js";

interface Booking {
  id: string;
```

```
  status: string;
}

interface Customer {
  name: string;
}

type BookingTransparentMethods = {
  updateStatus(status: string, reason: string): Promise<string>;
  customer: PromiseLike<TypedOop<Customer>>;
  rawCustomer: PromiseLike<Oop>;
};

const booking = transparentObject<Booking, BookingTransparentMethods>(object, {
  selectors: {
    updateStatus: "status:reason:",
  },
  objectSelectors: {
    customer: "customer",
  },
  oopSelectors: {
    rawCustomer: "customer",
  },
  snapshot: {
    id: "id",
    status: "status",
    customer: { selector: "customer", kind: "oop" },
    details: { selector: "details", kind: "dict", maxEntries: 100 },
  },
});

const before = await booking.status;
await booking.updateStatus("confirmed", "deposit received");
const customer = await booking.customer;
const rawCustomer = await booking.rawCustomer;
const payload = await booking.$snapshot();
```

Every non-$ property is an awaitable selector accessor. The same accessor can also be called like a method:

```
const status = await booking.status;
const sameStatus = await booking.status();
const rawStatus = await booking.status.oop();
```

This keeps the Python-like shape for reads while preserving JavaScript's async boundary.

Smalltalk-Style Bridge

`smalltalkBridge()` is the most transparent runtime layer. It is intended for explorers, scripts, notebooks, and migration work where gemstone-py's dynamic Smalltalk bridge style is useful. Globals are properties, selectors are properties, and selector dispatches can be awaited directly or called like functions.

```
import { smalltalkBridge } from "gemstone-js";

const st = smalltalkBridge(session);

const objectClassName = await st.Object.name;
const array = await st.Array.new_.transparent<
  Record<string, unknown>,
```

```
  { size: PromiseLike<number> }
>(3);
const arraySize = await array.size;
await st.UserGlobals.at_put_("GemStoneJsBridgeDemo", 42);
const storedValue = await st.UserGlobals.at_("GemStoneJsBridgeDemo");

await array.$release();
```

Selector property names are converted with `smalltalkSelectorForProperty()`:

- `size` sends `size`.
- `new_` sends `new:`, so `st.Array.new_(3)` sends `Array new: 3`.
- `at_put_` sends `at:put:`.
- `removeKey_ifAbsent_` sends `removeKey:ifAbsent:`.

When a selector cannot be represented clearly as a JavaScript property, keep it exact with `$send()`, `$sendOop()`, or `$sendObject()`:

```
const value = await st.$send("UserGlobals", "at:", "GemStoneJsBridgeDemo");
const rawOop = await st.Array.$sendOop("new:", 3);
const object = await st.Object.$sendObject("new");
```

Selector dispatches that return objects can create transparent proxies directly. Use `transparent()` when default selector inference is enough and `transparentWith()` when the proxy needs explicit selectors, object selectors, or snapshot policy.

```
const booking = await st.BookingRepository.find_.transparentWith<
  Booking,
  { updateStatus(status: string, reason: string): Promise<string> }
>(
  { selectors: { updateStatus: "status:reason:" } },
  "B-1001",
);

await booking.updateStatus("confirmed", "deposit received");
await booking.$release();
```

In production domain code, prefer generated wrappers or `Session.classRef()` for stable selector contracts. Use the bridge when dynamic Smalltalk ergonomics matter more than static discoverability.

Queued Assignment

Runtime property assignment is supported by the proxy. Because JavaScript setters cannot be `async`, assignments queue writes and callers flush them at a clear boundary.

```
(booking as unknown as { status: string }).status = "confirmed";
await booking.$flush();
await booking.$session.commit();
```

For fully typed application code, use `$assign()`:

```
await booking.$assign({ status: "confirmed" });
await booking.$session.commit();
```

If a queued write fails, `$flush()` or the next read reports the error. This avoids unhandled promise rejections from assignment syntax while still making write failures observable.

Cache and Refresh

By default, transparent property reads send to GemStone each time. Enable `cache: true` when a request wants local repeated reads:

```ts
const booking = transparentObject<Booking>(object, {
  cache: true,
});

const first = await booking.status;
const cached = await booking.status;
const fresh = await booking.status.refresh();

booking.$clearCache("status");
```

Cached values are per proxy and per session. They are not a global identity map and they do not replace GemStone transaction visibility.

Identity-Aware Proxy Reuse

`TransparentObjectMapper` reuses proxies by session and OOP so repeated wrapping of the same object can preserve local cache and UI identity.

```ts
import { TransparentObjectMapper } from "gemstone-js";

const mapper = new TransparentObjectMapper();
const first = mapper.wrap<Booking>(bookingHandle);
const second = mapper.wrap<Booking>(bookingHandle);

first === second; // true
```

Use a mapper inside request scopes, Explorer panes, or VS Code views where stable local wrapper identity is useful. Clear the mapper when the owning session closes.

Selector Configuration

Use explicit selector configuration whenever the JavaScript method name differs from the GemStone selector or the selector has more than one keyword.

```ts
import { mappedObject, type Oop, type TypedOop } from "gemstone-js";

interface Customer {
  name: string;
}

type BookingMapping = {
  updateStatus(status: string, reason: string): Promise<unknown>;
  customer(): Promise<TypedOop<Customer>>;
  customerOop(): Promise<Oop>;
};

const booking = mappedObject<Booking, BookingMapping>(object, {
  selectors: {
    updateStatus: "status:reason:",
  },
  objectSelectors: {
    customer: "customer",
  },
  oopSelectors: {
    customerOop: "customer",
  },
});

await booking.updateStatus("confirmed", "deposit received");
```

```
const customer = await booking.customer();
const rawCustomer = await booking.customerOop();
```

Use `objectSelectors` when the selector returns a live GemStone object that should be retained as `TypedOop<T>`. Use `oopSelectors` when the caller wants raw identity and will decide later whether to retain, inspect, or pass the OOP back into GemStone.

Snapshots

Snapshots are explicit readback payloads. They are useful at UI/API boundaries where plain data is safer than exposing a retained object handle.

```
const booking = mappedObject<Booking>(object, {
  snapshot: {
    id: "id",
    status: "status",
    customer: { selector: "customer", kind: "oop" },
    details: { selector: "details", kind: "dict", maxEntries: 100 },
  },
});

const payload = await booking.$snapshot();
```

Snapshot field kinds are:

- `value`: call `send()` and marshal the result back to JavaScript.
- `oop`: call `sendOop()` and return raw object identity.
- `object`: call `sendObject()` and return a retained `TypedOop<T>`.
- `dict`: call `sendOop()` and read a `StringKeyValueDictionary` back into a bounded JavaScript object.

Prefer snapshots for outbound data. Prefer retained handles for continued GemStone-side behavior.

`mappedObject()` API Reference

The runtime helper has one required argument, the retained object handle, plus an optional mapping description:

```
const booking = mappedObject<Booking, BookingMethods>(object, {
  selectors: {
    displayStatus: "status",
    updateStatus: "status:reason:",
  },
  setters: {
    setStatus: "status:",
  },
  objectSelectors: {
    customer: "customer",
  },
  oopSelectors: {
    rawCustomer: "customer",
  },
  snapshot: {
    id: "id",
    status: "status",
    customer: { selector: "customer", kind: "oop" },
    details: { selector: "details", kind: "dict", maxEntries: 100 },
  },
});
```

The options map directly to send behavior:

- `selectors`: Override selector inference for value-returning methods, for example `{ updateStatus: "status:reason:" }`.
- `setters`: Bind named setter methods to write selectors, for example `{ setStatus: "status:" }`.
- `objectSelectors`: Mark methods that return retained `TypedOop<T>` handles, for example `{ customer: "customer" }`.
- `oopSelectors`: Mark methods that return raw `Oop` identity, for example `{ rawCustomer: "customer" }`.
- `snapshot`: Configure default `$snapshot()` fields, for example `{ status: "status" }`.

Selector inference is intentionally narrow:

- `booking.status()` sends `status`.
- `booking.priority(3)` sends `priority:`.
- `booking.setStatus("held")` sends `status:`.
- `booking.updateStatus("held", "reason")` must be configured explicitly.

The proxy itself does not own a separate identity map. It delegates to the underlying retained object, so releasing either the proxy through `$release()` or the original `TypedOop<T>` releases the same retained handle.

Generated Selector Wrappers

Generated wrappers are the most ergonomic mapping layer for application code. The manifest or decorator source declares the GemStone class, selector, argument types, and return kind. The generated function keeps the selector send explicit while giving callers a typed API.

```
{
  "functions": [
    {
      "exportedName": "findBooking",
      "className": "Booking",
      "selector": "find:",
      "argNames": ["id"],
      "argTypes": ["string"],
      "returnType": "TypedOop<Booking>",
      "returnKind": "object"
    }
  ]
}
```

The generated output calls `classRef().sendObject()`:

```
export async function findBooking(
  session: Session,
  id: string,
): Promise<TypedOop<Booking>> {
  return session.classRef("Booking").sendObject("find:", id);
}
```

Use `returnKind: "value"` for simple marshalled values, `returnKind: "oop"` for raw object identity, and `returnKind: "object"` for retained typed handles.

Migration Path

Move toward mapping incrementally. A direct selector send:

```
const booking = await session
  .classRef<Booking>("BookingRepository")
  .sendObject("find:", "B-1001");

await booking.send("status:", "confirmed");
```

can first become a local proxy:

```ts
type BookingMethods = {
  setStatus(status: string): Promise<unknown>;
};

const bookingRef = mappedObject<Booking, BookingMethods>(booking, {
  setters: {
    setStatus: "status:",
  },
});

await bookingRef.setStatus("confirmed");
```

or a more transparent awaitable proxy:

```ts
const bookingRef = transparentObject<Booking>(booking);

const status = await bookingRef.status;
await bookingRef.$assign({ status: "confirmed" });
```

and later become generated source:

```ts
const bookings = new BookingRepository(session);
const bookingRef = await bookings.find("B-1001");

await bookingRef.setStatus("confirmed");
```

Each step keeps the same remote operation visible. The change is where selector names and return policies live: inline call site, proxy options, or committed generated code.

Explicit Dictionary Payloads

When you want to persist or pass a JavaScript object as structured data, convert it explicitly into a dictionary argument.

```ts
import {
  Session,
  objectToDictionaryArgument,
  scalarValueConverterRegistry,
} from "gemstone-js";

class BookingDraft {
  constructor(
    public id: string,
    public status: string,
    public requestedAt: Date,
  ) {}
}

const session = await Session.connect({
  valueConverters: scalarValueConverterRegistry(),
});

const draft = new BookingDraft("B-1001", "held", new Date("2026-05-19T12:00:00Z"));
const payload = objectToDictionaryArgument(draft);

const booking = await session
  .classRef<Booking>("Booking")
  .sendObject("createFromDictionary:", payload);
```

This mirrors the explicit `dataclass_to_dict()` style used in `gemstone-py`. Class instances become dictionary payloads only when the caller opts in.

Dictionary Mapping

GemStone `StringKeyValueDictionary` is the most direct bridge between JavaScript records and GemStone object storage. Use it when the shape is keyed, reviewable, and not worth modeling as a full GemStone class.

```
import { Session, type TypedOop } from "gemstone-js";

interface Booking {
  id: string;
  status: string;
}

const session = await Session.connect();
const booking = await session
  .classRef<Booking>("Booking")
  .sendObject("find:", "B-1001");

const dict = await session.dictionary({
  status: "held",
  tags: ["vip", "late-arrival"],
  limits: { guests: 2, bags: 3 },
});

await dict.setObject("booking", booking);
await dict.setAllValue({
  owner: "front-desk",
  priority: 3,
});

const status = await dict.requireValue("status");
const bookingAgain: TypedOop<Booking> = await dict.requireObject<Booking>("booking");
const limits = await dict.requireDict("limits");
const entries = await dict.items({ maxEntries: 50 });
const rawEntries = await dict.itemsOop({ maxEntries: 50 });
```

The value/object/raw naming is intentional:

- `getValue()`/`requireValue()` marshal the stored GemStone value back into a JavaScript value when possible.
- `getObject<T>()`/`requireObject<T>()` return retained `TypedOop<T>` handles for live GemStone objects.
- `getOop()`/`requireOop()` and `itemsOop()` preserve raw object identity.
- `getDict()`/`requireDict()` wrap nested dictionaries as `GsDict`.

When you only have a raw dictionary OOP, the session-level helpers expose the same mapping operations without creating a long-lived wrapper:

```
const dictOop = await session.dictionaryToOop({
  status: "held",
  updatedBy: "api",
});

await session.dictionarySetObject(dictOop, "booking", booking);
const values = await session.dictionaryEntries(dictOop, { maxEntries: 100 });
const handles = await session.dictionaryEntriesOop(dictOop, { maxEntries: 100 });
const mappedBooking = await session.dictionaryRequireObject<Booking>(dictOop, "booking");
```

For durable application state, dictionaries usually live under a persistent root or global. The root helper keeps the

same mapping choices visible:

```
import { PersistentRoot } from "gemstone-js";

const root = PersistentRoot.userGlobals(session);
const index = await root.setDict("BookingIndex", {
  kind: "booking-index",
  active: true,
});

await index.setObject("B-1001", booking);
await root.setObject("LastBooking", booking);

const savedIndex = await root.requireDict("BookingIndex");
const savedBooking = await savedIndex.requireObject<Booking>("B-1001");
const rootBooking = await root.requireObject<Booking>("LastBooking");
```

Dictionaries are a good mapping target for configuration, indexes, metadata, API payload snapshots, and small keyed aggregates. Prefer class references and typed object handles when the GemStone object has behavior that should remain on the GemStone side.

Root and Global Mapping

Roots and globals should usually store handles, dictionaries, or small payloads rather than hydrated JavaScript instances. This keeps transaction behavior visible and avoids implicit synchronization.

```
import { PersistentRoot, mappedObject, type TypedOop } from "gemstone-js";

interface Booking {
  id: string;
  status: string;
}

type BookingMethods = {
  setStatus(status: string): Promise<unknown>;
};

const root = PersistentRoot.userGlobals(session);
const bookingHandle: TypedOop<Booking> = await root.requireObject<Booking>("LastBooking");
const booking = mappedObject<Booking, BookingMethods>(bookingHandle, {
  setters: {
    setStatus: "status:",
  },
  snapshot: ["id", "status"],
});

await booking.setStatus("confirmed");
const payload = await booking.$snapshot();
```

Use this pattern when the root or global is an entry point into the GemStone object graph. Use `root.requireDict()` or `session.globalRequireDict()` when the entry itself is the mapped payload.

Mapping Relationships

Model relationships as selector methods that return handles, then decide at the call site whether to keep the handle or snapshot it.

```
import { mappedObject, type Oop, type TypedOop } from "gemstone-js";
```

```ts
interface Customer {
  name: string;
}

type BookingMapping = {
  customer(): Promise<TypedOop<Customer>>;
  customerOop(): Promise<Oop>;
};

const booking = mappedObject<Booking, BookingMapping>(bookingObject, {
  objectSelectors: {
    customer: "customer",
  },
  oopSelectors: {
    customerOop: "customer",
  },
});

const customer = await booking.customer();
const name = await customer.send<string>("name");
const rawCustomer = await booking.customerOop();
```

For UI payloads, snapshot the relationship by OOP, by nested dictionary, or by a small generated `CustomerRef.snapshot()` method. Avoid recursively hydrating an unbounded object graph unless the caller provides depth and item bounds.

Lifetime and Transactions

Mapping helpers do not change GemStone transaction rules:

- Reads observe the current session's transaction view.
- Writes through setters or `$set()` are normal GemStone selector sends.
- Call `session.commit()` to persist successful changes.
- Call `session.abort()` to discard uncommitted changes.
- Release retained object handles when they are no longer needed.

```ts
const booking = mappedObject<Booking, BookingMethods>(object, {
  setters: {
    setStatus: "status:",
  },
});

try {
  await booking.setStatus("confirmed");
  await booking.$session.commit();
} catch (error) {
  await booking.$session.abort().catch(() => undefined);
  throw error;
} finally {
  await booking.$release();
}
```

When an application already uses `RequestScope`, `TransactionScope`, or a pool, keep mapping objects scoped to the borrowed session. Do not cache a `mappedObject()` proxy beyond the lifetime of the session that created its underlying `TypedOop<T>`.

Value Converters

Converters run before built-in argument marshalling. They are best for scalar domain values that should cross the GemStone boundary in a consistent form.

```
import { Session, scalarValueConverterRegistry } from "gemstone-js";

const session = await Session.connect({
  valueConverters: scalarValueConverterRegistry(),
});

await session.classRef("AuditLog").send("recordAt:", new Date());
```

The built-in scalar registry stores `Date` values as ISO strings. Custom converters can add application-specific scalar wrappers while keeping object graph persistence explicit.

Readback and Inspection

For simple data structures, use bounded value readback:

```
const values = await session.arrayOopToValues(arrayOop, {
  maxDepth: 4,
  maxTotalItems: 500,
});

const dict = await session.dictionaryOopToObject(dictOop, {
  maxEntries: 100,
});
```

For live GemStone objects, prefer retained handles and inspection:

```
const object = session.typedOop<Booking>(bookingOop);
const summary = await object.inspect();
const dump = await object.dump({ maxDepth: 2, maxItems: 50 });
```

Bounded readback protects callers from unexpectedly large collections or deep object graphs. Inspection keeps object identity visible instead of pretending the remote object is a plain local value.

Mapping Manifest Roadmap

With `mappedObject()` available, the next object-mapping step should be connector-inspired code generation. The goal is to keep GemStone identity visible while giving application code a smaller, typed surface than raw selector sends or hand-written proxy options.

1. **Mapping manifest schema**

   Add a dedicated mapping manifest alongside the existing function-codegen manifest. Each mapped class should declare the GemStone class name, the TypeScript domain type, selectors, setters, repository selectors, and snapshot fields.

   ```
   {
     "$schema": "./schemas/object-mapping-manifest.schema.json",
     "classes": [
       {
         "name": "Booking",
         "gemStoneClass": "Booking",
         "typeName": "Booking",
         "refName": "BookingRef",
         "repository": {
           "className": "BookingRepository",
           "find": {
             "name": "find",
   ```

```
        "selector": "find:",
        "args": [{ "name": "id", "type": "string" }]
      }
    },
    "selectors": [
      { "name": "status", "selector": "status", "returnType": "string" }
    ],
    "setters": [
      {
        "name": "setStatus",
        "selector": "status:",
        "args": [{ "name": "status", "type": "string" }]
      }
    ],
    "snapshot": [
      { "name": "id", "selector": "id", "type": "string" },
      { "name": "status", "selector": "status", "type": "string" }
    ]
  }
 ]
}
```

2. **Generated `*Ref` classes**

   The generator should emit small reference classes that wrap `TypedOop<T>` or delegate to `mappedObject()`. Methods remain async and selector-backed; they do not become JavaScript properties.

```ts
export class BookingRef {
  constructor(readonly object: TypedOop<Booking>) {}

  get session(): Session {
    return this.object.session;
  }

  get oop(): Oop {
    return this.object.oop;
  }

  status(): Promise<string> {
    return this.object.send<string>("status");
  }

  async setStatus(status: string): Promise<this> {
    await this.object.send("status:", status);
    return this;
  }

  async snapshot(): Promise<BookingSnapshot> {
    return {
      id: await this.object.send<string>("id"),
      status: await this.status(),
    };
  }

  inspect() {
    return this.object.inspect();
  }
```

```
  release() {
    return this.object.release();
  }
}
```

3. **Repository helpers**

   Repository helpers should return typed refs rather than raw `TypedOop<T>` handles. They can wrap class-side selectors, persistent-root lookups, query helpers, or GemStone repository objects.

```
export class BookingRepository {
  constructor(readonly session: Session) {}

  async find(id: string): Promise<BookingRef> {
    const object = await this.session
      .classRef<Booking>("BookingRepository")
      .sendObject("find:", id);
    return new BookingRef(object);
  }
}
```

4. **Snapshot and dictionary helpers**

   Generated refs should support bounded `snapshot()` methods for UI/API payloads. Snapshot output should be plain data, not retained GemStone handles. Dictionary-backed snapshots should use the existing `dictionaryOopToObject()`, `GsDict`, and `PersistentRoot` helpers with `maxEntries` bounds.

5. **Explorer and VS Code mapping views**

   The Explorer and VS Code workbench can later read mapping manifests to show mapped classes, generated ref methods, repository helpers, and snapshot fields. This should be a tooling view over committed generated source, not a hidden runtime mapper.

This roadmap intentionally differs from a transparent JavaScript ORM. It is closer to the useful parts of the Pharo bridge connector model: reviewable class pair metadata, generated accessors, repository entry points, and inspectable mapping state. It avoids automatic local/GemStone synchronization until the typed ref and snapshot path is proven against live Stone workflows.

Production Checklist

Before relying on a mapping in application code:

- Keep selectors, setters, and snapshot fields in one reviewed place.
- Prefer generated wrappers or a small `*Ref` class when a mapping is reused.
- Use `transparentObject()` when you want `await booking.status` style reads and queued writes without generating a wrapper yet.
- Use `TransparentObjectMapper` only within the lifetime of its owning session or request scope.
- Use `objectSelectors` for relationship handles and `oopSelectors` for raw identity.
- Put `maxEntries`, `maxDepth`, and `maxItems` bounds on readback paths.
- Release retained handles with `await object.release()` or `await using`.
- Commit generated files and run `npm run examples:check` plus `npm run public-surface:check` when adding public mapping helpers.
- Run `GS_RUN_LIVE=1 npm run test:live` before treating a new mapping as Stone-compatible.

What Is Not Automatic

`gemstone-js` does not currently provide:

- synchronous `booking.status` property dispatch to GemStone selectors
- TypeScript-native asynchronous assignment syntax; runtime assignment is queued and must be observed with `$flush()`, while typed code can use `$assign()`
- automatic JS class hydration from arbitrary GemStone instances

- a session-wide identity map for local wrapper instances
- automatic persistence of arbitrary JavaScript object graphs
- lazy slot loading hidden behind synchronous property access
- bidirectional local/GemStone synchronization through connector rows

Those features can be useful in narrow domains, but they also create sharp edges around transactions, remote latency, conflict handling, and object identity. The current object-mapping API keeps those boundaries reviewable.